

Convolutional Neural Network

Hyoung Joon, Kim

June 3, 2026

Contents

1	Introduction	2
2	Main Ideas	3
2.1	Invariance and Equivariance	3
2.2	Convolution	3
2.2.1	1D Convolution Operation	3
2.2.2	Padding, Stride, Kernel Size, and Dilation	4
2.2.3	Channels	5
2.2.4	Receptive Fields	5
2.2.5	2D Convolution Operation	5
2.2.6	Toeplitz Matrix	5
2.3	Downsampling, Upsampling, and Changing the Number of Channels	6
2.3.1	Downsampling	6
2.3.2	Upsampling	6
2.3.3	Changing the Number of Channels	7
3	Applications	8

1 Introduction

Convolutional Neural Network (CNN) is used to process mainly the image data. Processing image directly by just flattening image data leads to enormous amount of calculation, and it loses the local relevance between pixels. CNN solves this problem. CNN has some remarkable properties:

1. Invariance and equivariance about geometric transformations;
2. Exploits local relevance of pixels. This is implemented via convolutional filters;
3. Shares parameters, which leads to fewer parameters compared to fully connected layer;

2 Main Ideas

2.1 Invariance and Equivariance

This section show the idea of mathematically precise meaning of consistency under geometric transformations.

Definition 2.1.1 (Invariance)

Function $f[x]$ of an image x is **invariant** to a transformation $t[x]$ if

$$f[t[x]] = f[x]$$

In other word, the output is same regardless of the transformation $t[x]$. Networks for image classification should maintain this property. That is, even if the image of a cat is shifted or scaled, the CNN must still classify it as cat.

Definition 2.1.2 (Equivariance)

Function $f[x]$ of an image x is **equivariant** (or covariant) to a transformation $t[x]$ if

$$f[t[x]] = t[f[x]]$$

That is, if the input has gone through some geometric transformation, the output must be same as the original output with same geometric transformation applied. Networks for image segmentation should maintain this property.

2.2 Convolution

Now we discuss about convolution operation, the heart of CNN.

2.2.1 1D Convolution Operation

First, we discuss about 1D convolution operation. We can think of this as just simply calculating the weighted average of nearby inputs.

Definition 2.2.1 (1D Convolution)

Let x be a vector, and w be a weight vector, which is called **kernel**. Then, the 1D convolution operation $*$, whose result is also a vector, is defined as follows:

$$(x * w)_n = \sum_{m=-\infty}^{\infty} x_m w_{n-m}$$

Actually in practice, we use another definition, known as cross-correlation.

Definition 2.2.2 (1D cross-correlation)

Let $\mathbf{x}$ be a vector, and $\mathbf{w}$ be a kernel. Then, the 1D cross-correlation $\star$, whose result is also a vector, is defined as follows:

$$(\mathbf{x} \star \mathbf{w})_n = \sum_{m=-\infty}^{\infty} \mathbf{x}_m \mathbf{w}_{m-n}$$

From now on, we will refer to cross-correlation as convolution.

Example 2.2.1 (Convolution Operation(cross-correlation))

Let $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{w} \in \mathbb{R}^3$. Let $\mathbf{z} = \mathbf{x} \star \mathbf{w}$. Then, $z_i = w_1 x_{i+1} + w_2 x_{i+2} + w_3 x_{i+3}$. You can think of this as a window passing through the data, making a dot product. Regard the value of data as 0 where the index is out of range. This is called a padding, which we will discuss later.

2.2.2 Padding, Stride, Kernel Size, and Dilation

Now we present definitions of padding, stride, kernel size and dilation, which alters the convolution operation a bit.

Definition 2.2.3 (Padding, Stride, Kernel Size, and Dilation)

- **Padding** is used to deal with the data index out of range. The most common method is **zero-padding**, where we fill the undefined value with 0. We also can regard the input as circular or reflecting it at the boundaries. Or, we can just ignore the convolution result when the data index is out of range, introducing no extra information. This is called **valid convolutions**, but this method decreases the representation in size.
- **Stride** is used to control the step of kernel moving through the data. The Definition 2.2.2 or 2.2.1 uses stride 1. If we use stride k , the next element z_{i+1} of Example 2.2.1 is $z_{i+1} = w_1 x_{i+1+k} + w_2 x_{i+2+k} + w_3 x_{i+3+k}$
- **Kernel size** is just the dimension of the weight vector used as kernel. The kernel in Example 2.2.1 has kernel size 3. If the kernel size grows, then we can deal with larger area of the data at once, using more informations. However, this results in more weights. This leads to the idea of dilated or atrous convolutions.
- **Dilation** is to make the kernel sparse. That is, some of the weight value of the kernel is 0, which enables bigger kernel size with not-so-much weights. For example, if we use **dilation rate** 2, which means the next non-zero weight comes after two element, to Example 2.2.1, the result is $z_i = w_1 x_{i+1} + w_2 x_{i+3} + w_3 x_{i+5}$. Here we did not actually increased the kernel size, but changed the data being multiplied.

We can calculate the size of the output result of the convolution by the following formula.

Theorem 2.2.1 (Resulting Size of the 2D Convolution)

Let the input data have size $J \times K$ and have P padding. Let the kernel have size $M \times M$ and have stride S . Then the resulting size of the convolution is

$$\left\lfloor \frac{J + 2P - M}{S} - 1 \right\rfloor \times \left\lfloor \frac{K + 2P - M}{S} - 1 \right\rfloor$$

2.2.3 Channels

Definition 2.2.4 (Channel)

Channel is a hidden layer created by the convolution, which is also referred to as **feature map**.

There can be more than one channel, if we use more than one convolutional kernel filters. In practice, we produce more than one channel since only using one convolutional operation would lost some informations. Input variables also can have more than one channel. For example, an image data would have three to four channels, since it has RGB values or RGBA values.

2.2.4 Receptive Fields

Definition 2.2.5 (Receptive Fields)

The **receptive field** of a hidden layer is the region of the original input that feeds into it.

Example 2.2.2 (Size of Receptive Field of Hidden Layer)

Let's assume that we apply convolution operation two times, with same kernel with kernel size 3. Then, each element of the first hidden layer would have been calculated with three input data. So, the size of the receptive field of the first hidden layer is 3. Each element of the second hidden layer would have been calculated with five input data, since

$$\begin{aligned} z_i^{(2)} &= w_1 z_{i+1}^{(1)} + w_2 z_{i+2}^{(1)} + w_3 z_{i+3}^{(1)} \\ &= w_1^2 x_{i+2} + 2w_1 w_2 x_{i+3} + (2w_1 w_3 + w_2^2) x_{i+4} + 2w_2 w_3 x_{i+5} + w_3^2 x_{i+6} \end{aligned}$$

as you can see. The size of receptive field is 5.

2.2.5 2D Convolution Operation

2D convolution operation is similar to 1D convolution operation. The definitions discussed above can be extended to 2D convolution operations easily.

Definition 2.2.6 (2D Convolution Operation)

Let $\mathbf{X} \in \mathbb{R}^{n \times m}$, and let $\mathbf{W} \in \mathbb{R}^{k \times l}$. Then,

$$(\mathbf{X} * \mathbf{W})_{i,j} = \sum_{p=1}^k \sum_{q=1}^l X_{i+p,j+q} W_{p,q}$$

This operation can be also thought as weighted average over nearby pixels.

2.2.6 Toeplitz Matrix

Convolution operation can be expressed in matrix multiplication form, due to the famous **Fundamental Theorem of Linear Algebra**, which states that any linear operation can be expressed in matrix form. The convolution has matrix expression, where the matrix is of special kind, called **Toeplitz matrix**.

Definition 2.2.7 (Toeplitz Matrix)

ADD THE DEFINITION OF TOEPLITZ MATRIX

Example 2.2.3 (Matrix Representation of Convolution Operation)

ADD THE EXAMPLE OF MATRIX REPRESENTATION OF CONVOLUTION

2.3 Downsampling, Upsampling, and Changing the Number of Channels

2.3.1 Downsampling

Downsampling is to decrease the size of 2D input representations. There are some methods to do this:

1. **Increase the stride.** If we use stride of size 2, then the result hidden layer is scaled down to half.
2. **Max Pooling.** Max pooling picks the maximum value from 2×2 input values. This induces some invariance to translation; moving about one pixel won't change the result of max pooling.
3. **Average Pooling.** Average pooling calculates the average value of 2×2 input values and use it as a value of downsampled data.

Downsampling is applied to each channels, so after the downsampling, the number of channels won't change.

2.3.2 Upsampling

Upsampling is to scale up the resolution of the data.

1. **Duplicate the data.** This is the simplest way to double the resolution.
2. **Max Unpooling.** Max unpooling is used when we previously used a max pooling operation for downsampling. We distribute the value to the position where it originated from.
3. **Bilinear Interpolation.** Fill the missing values between points with known values using bilinear interpolation.
4. **Transposed Convolution.** Using stride 2 kernel decrease the size to half. Transposed convolution does this in kind of opposite way, doubling the size. The reason that this is called transposed convolution is because matrix representation of the operation is same as the transpose of that of ordinary convolution operation. The kernel used in transposed convolution need not be the same as the one used in the preceding convolution.

Remark 2.3.1 (Transposed Convolution)

Transposed convolution can be implemented via ordinary convolution with dilation. The point is that the dilation is applied to the input data, not the kernel. Let's assume that we used stride s kernel in convolution operation. Then, to retrieve the data to the original size using transposed convolution, we take the following step:

1. Add $s - 1$ zeros between each input data.
2. Flip the kernel (if you are using same kernel used in convolution operation).
3. Do the original convolution.

2.3.3 Changing the Number of Channels

To change the number of channel itself, go through the following step.

1. Calculate the weighted sum of each channel element at the same position.
2. Apply activation functions if one wants.
3. We can repeat this as much as we want, decreasing or increasing the number of channels.

If the input has C_i channels and the output has C_o channels, then there are $1 \times 1 \times C_i \times C_o$ weights. Hence, this is known as 1×1 convolution. This is same as making fully connected layer at the input channels with output of size same as the number of output channels we want.

3 Applications